## Your grade: 80%

Your latest: **80%** • Your highest: **80%** • To pass you need at least 80%. We keep your highest score.

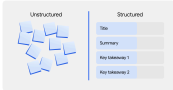
Next item →

### Intelligent information processor

## Overview

In this activity, you'll build a script that uses the Google Gemini API to read content from a web article and transform that information into structured data. Structured data is information organized in a predictable format. This skill is critical for building reliable AI applications that can process information from the web. The final organized data format is often called agent-ready because it can be easily used by other systems.

**From unstructured to structured data**




## Scenario

Imagine you are a developer for a company that builds AI assistants. A client wants their assistant to read any online news article and provide a quick summary. For the summary to be used automatically by their system, the information must *always* be in the same format: a title, a short summary, and a list of three key points. Your task is to build the part of the system that gets and structures this information.

## Prerequisites

You must have a secure way to access your API key.

**Important security reminder:** The API key must never be written directly in your code. The best practice is to read the `GEMINI_API_KEY` from your computer's environment.

## Steps

### Step 1: Install SDK and configure environment

This step ensures your environment is ready to start coding.

1. **Install the SDK:**
   - **For Python:** Run: `pip install google-genai`
   - **For JavaScript/Node.js:** Run: `npm install @google/genai`
2. **Configure your API key:** The best practice is to use an environment variable. Do not write your key directly in your code.

**Common mistake:** Writing your secret API key directly into the code file instead of using an environment variable.

1. **You are setting up a script to process news articles. You decide to save your Gemini API key as an environment variable on your machine instead of typing it directly into the script file. What is the primary security advantage of this approach? Select the best answer.** 1 / 1 point

- ○ It allows the Gemini API to automatically access and scan the developer's private local documents.
- ○ It makes the API respond faster because the key is already "pre-loaded" into the computer's memory.
- ○ It is the only way to enable the URL Context tool in the Gemini SDK.
- ● It prevents the secret key from being exposed if the code is shared, uploaded to a public repository, or viewed by others.

✓ Nice work

Correct! Environment variables keep sensitive credentials separate from the logic of your code, ensuring secrets aren't accidentally "leaked" when the file is moved or shared.

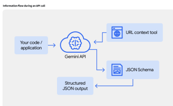
### Step 2: Define the output structure with a JSON schema

You will define the structure you want the Gemini model to return. This schema tells the model exactly what information to include in its answer.

```
# Define the required output structure for the article analysis
ARTICLE_SCHEMA = {
    "type": "object",
    "properties": {
        "title": {"type": "string"},
        "summary": {"type": "string"},
        "key_takeaways": {
            "type": "array",
            "items": {"type": "string"},
            "minItems": 3,
            "maxItems": 3
        }
    },
    "required": ["title", "summary", "key_takeaways"]
}
```

**Common mistake:** Forgetting to include `"required"` fields, which can make the output unreliable.

**Information flow during an API call**




2. **A data analyst is using the Gemini API to extract information from company reports. The API returns text, but the format is different every time, making it hard to use in their automated scripts. To solve this, they need the API to always return data with specific fields like "report_title", "publication_date", and "key_findings". Which feature should they use to enforce this consistent structure? Select the best answer.** 1 / 1 point

- ● A JSON Schema to define the required output format
- ○ A longer, more detailed text prompt
- ○ The Google Search built-in tool
- ○ An environment variable to store the format

✓ Nice work

Correct! A JSON Schema tells the model exactly what structure and data types to use in its response, ensuring the output is consistent and predictable.

### Step 3: Write the code to call the API

Now, write the code to make the API call. You must pass both the URL Context tool and the JSON Schema in the request configuration.

Use this sample article for your test from the [Google Developers Blog.](#)

- Python starter code (The full solution is available in the Exemplar):
  - **Action:** Copy the template below and fill in lines marked with `# YOUR CODE HERE`.

**Note:** With the Gemini 3 series, you can combine **Structured Outputs** with **built-in tools** (e.g., Google Search, URL Context, Code Execution) in a **single model call**. This means you don't need to run a separate follow-up request just to reformat a tool-grounded answer into your JSON schema—provide the `tools` config and the response schema together and the model can return schema-compliant JSON in one step as referenced in the [Gemini 3 Developer Guide.](#)

```
from google import genai
import os
import json
from google.genai import types


from dotenv import load_dotenv

# Load variables from .env into the environment
load_dotenv()
# Configure the API key
genai.configure(api_key=os.environ["GEMINI_API_KEY"])

# The URL of the article to analyze
article_url = "https://developers.googleblog.com/en/new-gemini-api-updates-for-gemini-3/"

# (The article_schema defined in Step 2 must be available here)

# CREATE CLIENT (new google-genai style)
api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    print("FATAL ERROR: GEMINI_API_KEY environment variable not set.")
    raise SystemExit(1)

  # YOUR CODE HERE: Create Client Command

# --- FETCH ARTICLE SUMMARY USING URL CONTEXT (TOOLS)
def fetch_article_summary(article_url: str) -> str:
    """
    Use URL Context to read the article and return a concise summary.
    This call uses tools, so we DO NOT request JSON here.
    """
    prompt = (
        # YOUR CODE HERE: Prompt the AI to use URL Context to read the article at the URL below.

    )



# Make the generation request
```

3. **An AI agent is designed to help users find and summarize recipes from cooking websites. The agent first uses the URL Context tool to get the recipe text from a given URL. To be useful, the agent needs to separate the ingredients from the cooking instructions so it can display them differently. What is the most reliable way to achieve this? Select the best answer.** 1 / 1 point

- ● Combine the URL Context tool with a JSON Schema that defines separate fields for ingredients and instructions.
- ○ Use string splitting functions in the code to search for keywords like "Ingredients:".
- ○ Ask the model to "please separate the ingredients and instructions" in the prompt.
- ○ Only use websites that have a "Print Recipe" button.

✓ Nice work

Correct! This is the most robust solution. The tool provides the content, and the JSON Schema forces the model to process that content into a reliable, structured format that the agent can easily use.

4. **A developer is building an application to summarize the main points of breaking news articles as they are published. The application needs the most current information available on the web, often from within the last hour. Which Gemini API built-in tool is best suited for this task? Select the best answer.** 0 / 1 point

- ○ A custom function for web scraping
- ○ The Google Search tool
- ○ The base Gemini Pro model without any tools
- ● The URL Context tool

⊗ Try again

Not quite. The URL Context tool is perfect for analyzing content at a specific, known URL, but it cannot discover new articles on its own.

### Step 4: Check your work and evaluate

Run your script. Does your output look correct?

### Success checklist

- Your script runs without any errors.
- The output is a clean JSON object, not plain text.
- The JSON object contains the required keys: `title`, `summary`, and `key_takeaways`.
- The `key_takeaways` value is an array containing three strings.

## Example output

Your final output should look similar to this (the exact text will vary slightly):

```
--- Raw Summary from URL Context (BEFORE) ---
The post ("Gemini 3 Flash is now available in Gemini CLI", Dec 17, 2025) announces that Gemini 3 Flash can now be selected inside Gemini CLI for fast, high-frequency terminal workflows. It claims 78% SWE-bench Verified for agentic coding (outperforming the 2.5 series and even Gemini 3 Pro) and positions Flash as a quality-vs-cost/speed option, available in preview at under one-quarter the cost of Gemini 3 Pro.

It explains availability: most paid-tier Gemini CLI users can access Gemini 3 Pro + Gemini 3 Flash (Google AI Pro/Ultra non-business customers, users with paid API keys via Google AI/Vertex, and eligible Gemini Code Assist users enabled by admins). For free tier users, the waitlist has been onboarded and broader access is rolling out gradually.

Setup steps: update Gemini CLI to v0.21.1+ (npm install -g @google/gemini-cli@latest), enable Preview features in /settings, then use /model to select Gemini 3. The post highlights CLI auto-routing (reserve Pro for hardest tasks) or manual model selection, arguing Flash's improved reasoning can handle tasks that previously needed Pro.
It includes examples of "agentic coding" workflows: generating a more complex app/prototype (3D voxel demo), extracting the key actionable request from a very large pull request thread (1,000 comments), and generating/fixing an asyncio load-testing script for a Cloud Run service after runtime errors.

```

- **Agent-ready:** Structured output ensures data is "agent-ready"—meaning it is formatted so that other software, agents, or APIs can easily consume and act upon the information without ambiguity.

```
--- Structured Article Data (AFTER) ---
{
  "title": "Gemini 3 Flash is now available in Gemini CLI",
  "summary": "Google announces that Gemini 3 Flash is now selectable in Gemini CLI for fast, high-frequency terminal workflows. The post highlights Gemini 3 Flash's agentic coding performance (including a reported 78% SWE-bench Verified score), positions it as a strong quality-cost-speed option (preview pricing stated as less than one-quarter the cost of Gemini 3 Pro), and explains who gets access (most paid-tier users, with gradual rollout for free-tier). It also provides setup steps: update Gemini CLI to v0
  "key_takeaways": [
    "Gemini 3 Flash is now available in Gemini CLI, targeting high-frequency terminal workflows and emphasizing strong agentic coding performance.",
    "Access is broadly available for most paid-tier Gemini CLI users (and rolling out for free-tier), with guidance on how different user groups can enable Gemini 3 models.",
    "To use it, upgrade Gemini CLI to v0.21.1+, toggle Preview features in /settings, then select Gemini 3 via /model (or rely on auto-routing)."
  ]
}
{
  "title": "Gemini 3 Flash is now available in Gemini CLI",
  "summary": "Google announces that Gemini 3 Flash is now selectable in Gemini CLI for fast, high-frequency terminal workflows. The post highlights Gemini 3 Flash's agentic coding performance (including a reported 78% SWE-bench Verified score), positions it as a strong quality-cost-speed option (preview pricing stated as less than one-quarter the cost of Gemini 3 Pro), and explains who gets access (most paid-tier users, with gradual rollout for free-tier). It also provides setup steps: update Gemini CLI to v0
  "key_takeaways": [
    "Gemini 3 Flash is now available in Gemini CLI, targeting high-frequency terminal workflows and emphasizing strong agentic coding performance.",
    "Access is broadly available for most paid-tier Gemini CLI users (and rolling out for free-tier), with guidance on how different user groups can enable Gemini 3 models.",
    "To use it, upgrade Gemini CLI to v0.21.1+, toggle Preview features in /settings, then select Gemini 3 via /model (or rely on auto-routing)."
  ]
}
--------------------------------------

```

### Key point

**Reliable data:** Combining a Tool (like URL Context) with a JSON Schema is the most robust way to guarantee the model's output is reliable and predictable, enabling automated processing.

5. **An application is considered "agent-ready" because it outputs structured JSON instead of a natural language paragraph. Why is this structured format preferred when building complex AI systems? Select the best answer.** 1 / 1 point

- ○ JSON text is generally easier and faster for a human to read than a standard paragraph.
- ● Structured data can be easily parsed by other software programs, allowing the output to trigger follow-up actions automatically.
- ○ JSON is the only data format that the Gemini model is capable of generating.
- ○ Using JSON allows the AI to browse the web without needing any additional tools like URL Context.

✓ Nice work

Correct! Because the data is in a predictable format with labeled fields, other parts of an application (like a database or an email bot) can reliably "grab" the specific info they need.

### Exemplar solution

```
import os
import json
from dotenv import load_dotenv
from google import genai

# Load variables from .env into the environment
load_dotenv()

# ---  JSON SCHEMA FOR STRUCTURED OUTPUT ---
ARTICLE_SCHEMA = {
    "type": "object",
    "properties": {
        "title": {"type": "string"},
        "summary": {"type": "string"},
        "key_takeaways": {
            "type": "array",
            "items": {"type": "string"},
            "minItems": 3,
            "maxItems": 3,
        },
    },
    "required": ["title", "summary", "key_takeaways"],
}

# ---  CREATE CLIENT (new google-genai style) ---
api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    print("FATAL ERROR: GEMINI_API_KEY environment variable not set.")
    raise SystemExit(1)

client = genai.Client(api_key=api_key)

# The URL of the article to analyze
article_url = "https://developers.googleblog.com/gemini-3-flash-is-now-available-in-gemini-cli/"


# ---  FETCH ARTICLE SUMMARY USING URL CONTEXT (TOOLS) ---
def fetch_article_summary(article_url: str) -> str:
    """
    Use URL Context to read the article and return a concise summary.
```

### Next steps

- **Optional challenge (coding):** Integrate a new field into your schema and update your prompt to extract information like the article's **sentiment** (Positive, Negative, or Neutral).
- **Explore tool chaining (future learning):** Research how to use the Google Search tool first, and then pass the search results (like a list of URLs) into the URL Context tool for analysis.